Lab # 04

Finite State Machines using VHDL

Digital System Design

Name: Muhammad Ahmad
Roll No: FA23-BCE-113
Section: BCE-6A
Date: March 28, 2026

Department of Computer Engineering
HITEC University, Taxila

Contents

1 Moore vs Mealy Machines: A Comparative Analysis

1.1 Moore Machine

- **Output depends ONLY on current state**
- Output changes synchronously with state transitions
- Generally requires more states than Mealy
- Output is more stable (no glitches from input changes)
- Easier to design and debug

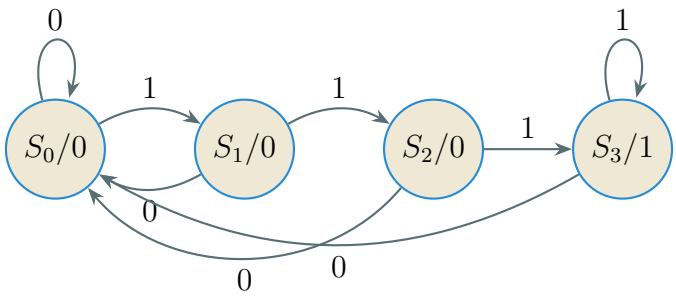

Figure 1: Moore Machine for "111" Detection (Output delayed by one cycle)

1.2 Mealy Machine

- **Output depends on current state AND current input**
- Output changes immediately when input changes (asynchronous)
- Generally requires fewer states than Moore
- Faster response (output appears same cycle as input)
- More prone to glitches if input is noisy

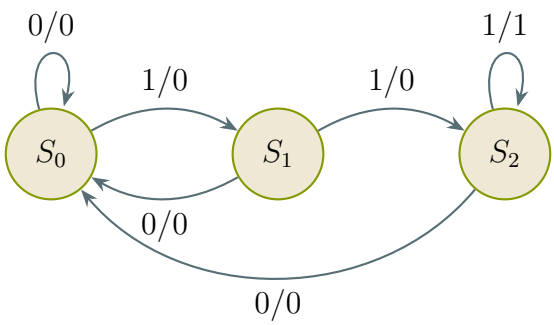

Figure 2: Mealy Machine for "111" Detection (Immediate output)

1.3 Comparison Table

Feature	**Moore**	**Mealy**
Output depends on	State only	State + Input
Number of states	More	Fewer
Response time	One clock delay	Immediate
Output stability	More stable	May have glitches
Complexity	Simpler	More complex

2 Pre-Lab Part 1: Up/Down 4-Bit Counter (Moore FSM)

2.1 FSM State Diagram

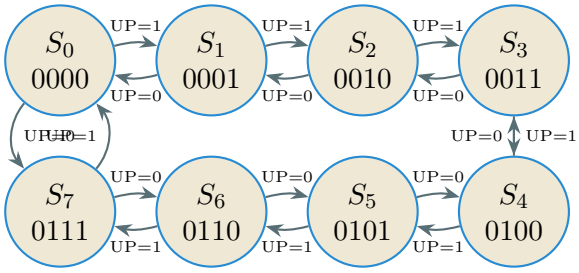

Figure 3: Up/Down Counter FSM (Moore Machine)

2.2 Design Approach

This is a **Moore Machine** because:

- The output (count) depends **only** on the current state
- Each state has a fixed output value regardless of input
- The UP_DOWN input only affects the next state, not the output

2.3 VHDL Implementation

up_down_counter.vhd

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity up_down_counter is
    port (
        clk     : in std_logic;
        clr     : in std_logic;
        up_down : in std_logic;
        count   : out std_logic_vector (3 downto 0)
    );
end up_down_counter;

architecture bhv of up_down_counter is
    type state_type is (s0,s1,s2,s3,s4,s5,s6,s7);
    signal current_state, next_state : state_type;
begin

------------- Process 1: State Register -------------
    process (clk, clr)
    begin
        if clr = '1' then
```

```
            current_state <= s0;
        elsif rising_edge(clk) then
            current_state <= next_state;
        end if;
    end process;

------------- Process 2: Next State Logic -------------
    process (current_state, up_down)
    begin
        case current_state is
            when s0 =>
                if up_down = '1' then
                    next_state <= s1;
                else
                    next_state <= s7;
                end if;
            when s1 =>
                if up_down = '1' then
                    next_state <= s2;
                else
                    next_state <= s0;
                end if;
            when s2 =>
                if up_down = '1' then
                    next_state <= s3;
                else
                    next_state <= s1;
                end if;
            when s3 =>
                if up_down = '1' then
                    next_state <= s4;
                else
                    next_state <= s2;
                end if;
            when s4 =>
                if up_down = '1' then
                    next_state <= s5;
                else
                    next_state <= s3;
                end if;
            when s5 =>
                if up_down = '1' then
                    next_state <= s6;
                else
                    next_state <= s4;
                end if;
            when s6 =>
                if up_down = '1' then
                    next_state <= s7;
                else
                    next_state <= s5;
```

```
                end if;
            when s7 =>
                if up_down = '1' then
                    next_state <= s0;
                else
                    next_state <= s6;
                end if;
        end case;
    end process;

------------- Process 3: Output Logic (Moore) -------------
    process(current_state)
    begin
        case current_state is
            when s0 => count <= "0000";
            when s1 => count <= "0001";
            when s2 => count <= "0010";
            when s3 => count <= "0011";
            when s4 => count <= "0100";
            when s5 => count <= "0101";
            when s6 => count <= "0110";
            when s7 => count <= "0111";
        end case;
    end process;

end bhv;
```

3 Pre-Lab Part 2: "111" Sequence Detector (Mealy FSM)

3.1 FSM State Diagram

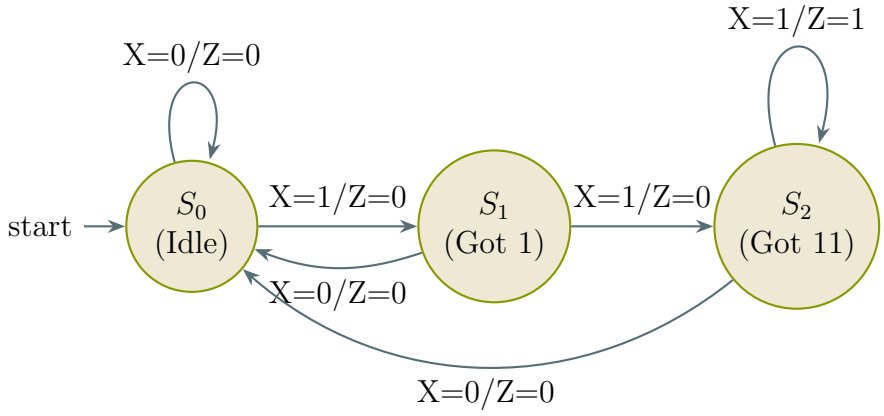

Figure 4: "111" Sequence Detector - Mealy Machine

3.2 State Transition Table

Current State	Input X	Next State	Output Z
S0 (Idle)	0	S0	0
S0 (Idle)	1	S1	0
S1 (Got 1)	0	S0	0
S1 (Got 1)	1	S2	0
S2 (Got 11)	0	S0	0
S2 (Got 11)	1	S2	**1**

3.3 Why Mealy?

- Output Z=1 **immediately** when third '1' arrives (same clock cycle)
- Overlapping detection: stays in S2 after detection to catch consecutive "111" patterns
- Faster response than Moore (no extra state needed for output)

3.4 VHDL Implementation

seq_detector_111_mealy.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity seq_detector_111_mealy is
    port (
```

```
        clk : in std_logic;
        rst : in std_logic;
        x   : in std_logic;
        z   : out std_logic
    );
end seq_detector_111_mealy;

architecture bhv of seq_detector_111_mealy is
    type state_type is (s0, s1, s2);
    signal current_state, next_state : state_type;
begin

------------- Process 1: State Register -------------
    process (clk, rst)
    begin
        if rst = '1' then
            current_state <= s0;
        elsif rising_edge(clk) then
            current_state <= next_state;
        end if;
    end process;

------------- Process 2: Next State Logic -------------
    process (current_state, x)
    begin
        case current_state is
            when s0 =>
                if x = '1' then
                    next_state <= s1;
                else
                    next_state <= s0;
                end if;
            when s1 =>
                if x = '1' then
                    next_state <= s2;
                else
                    next_state <= s0;
                end if;
            when s2 =>
                if x = '1' then
                    next_state <= s2;  -- Stay for overlapping
                else
                    next_state <= s0;
                end if;
        end case;
    end process;

------------- Process 3: Output Logic (Mealy) -------------
-- Output depends on BOTH current_state AND input x
    process (current_state, x)
    begin
```

```
        case current_state is
            when s0 => z <= '0';
            when s1 => z <= '0';
            when s2 =>
                if x = '1' then
                    z <= '1';  -- Immediate output!
                else
                    z <= '0';
                end if;
        end case;
    end process;

end bhv;
```

4 In-Lab Task 1: Even-Odd Up/Down Counter

4.1 FSM State Diagram

The counter has two modes:

- **Even Mode**: Cycles through 0, 2, 4, 6, 8, 10, 12, 14
- **Odd Mode**: Cycles through 1, 3, 5, 7, 9, 11, 13, 15

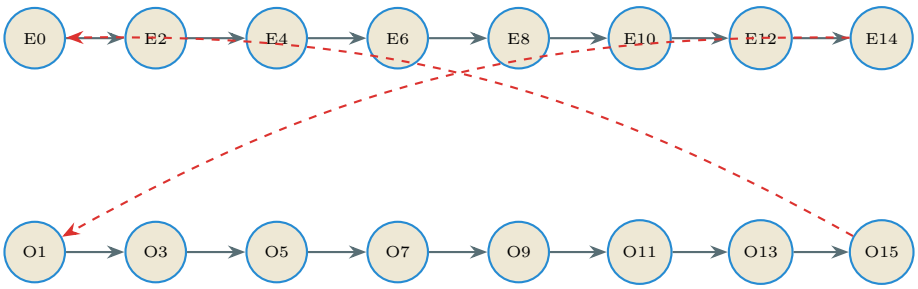

Figure 5: Even-Odd Counter FSM (Moore Machine) - UP direction shown

Note: DOWN direction reverses all transitions. Dashed red arrows show mode switching.

4.2 VHDL Implementation

even_odd_counter.vhd

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity even_odd_counter is
    port (
        clk   : in std_logic;
        rst   : in std_logic;
        cen   : in std_logic;  -- Counter enable
        dir   : in std_logic;  -- Direction: 1=UP, 0=DOWN
        count : out std_logic_vector(3 downto 0)
    );
end even_odd_counter;

architecture bhv of even_odd_counter is
    type state_type is (e0, e2, e4, e6, e8, e10, e12, e14,
                        o1, o3, o5, o7, o9, o11, o13, o15);
    signal current_state, next_state : state_type;
begin

------------- Process 1: State Register -------------
    process (clk, rst)
    begin
        if rst = '1' then
```

```
            current_state <= e0;
        elsif rising_edge(clk) then
            if cen = '1' then
                current_state <= next_state;
            end if;
        end if;
    end process;

------------- Process 2: Next State Logic -------------
    process (current_state, dir)
    begin
        case current_state is
            -- Even Mode States
            when e0 =>
                if dir = '1' then next_state <= e2;
                else next_state <= o15;
                end if;
            when e2 =>
                if dir = '1' then next_state <= e4;
                else next_state <= e0;
                end if;
            when e4 =>
                if dir = '1' then next_state <= e6;
                else next_state <= e2;
                end if;
            when e6 =>
                if dir = '1' then next_state <= e8;
                else next_state <= e4;
                end if;
            when e8 =>
                if dir = '1' then next_state <= e10;
                else next_state <= e6;
                end if;
            when e10 =>
                if dir = '1' then next_state <= e12;
                else next_state <= e8;
                end if;
            when e12 =>
                if dir = '1' then next_state <= e14;
                else next_state <= e10;
                end if;
            when e14 =>
                if dir = '1' then next_state <= o1;
                else next_state <= e12;
                end if;
            -- Odd Mode States
            when o1 =>
                if dir = '1' then next_state <= o3;
                else next_state <= e14;
                end if;
            when o3 =>
```

```
                if dir = '1' then next_state <= o5;
                else next_state <= o1;
                end if;
            when o5 =>
                if dir = '1' then next_state <= o7;
                else next_state <= o3;
                end if;
            when o7 =>
                if dir = '1' then next_state <= o9;
                else next_state <= o5;
                end if;
            when o9 =>
                if dir = '1' then next_state <= o11;
                else next_state <= o7;
                end if;
            when o11 =>
                if dir = '1' then next_state <= o13;
                else next_state <= o9;
                end if;
            when o13 =>
                if dir = '1' then next_state <= o15;
                else next_state <= o11;
                end if;
            when o15 =>
                if dir = '1' then next_state <= e0;
                else next_state <= o13;
                end if;
        end case;
    end process;

------------- Process 3: Output Logic (Moore) -------------
    process (current_state)
    begin
        case current_state is
            when e0  => count <= "0000";
            when e2  => count <= "0010";
            when e4  => count <= "0100";
            when e6  => count <= "0110";
            when e8  => count <= "1000";
            when e10 => count <= "1010";
            when e12 => count <= "1100";
            when e14 => count <= "1110";
            when o1  => count <= "0001";
            when o3  => count <= "0011";
            when o5  => count <= "0101";
            when o7  => count <= "0111";
            when o9  => count <= "1001";
            when o11 => count <= "1011";
            when o13 => count <= "1101";
            when o15 => count <= "1111";
        end case;
```

```
    end process;

end bhv;
```

5 In-Lab Task 2: "1101" Sequence Detector

5.1 Understanding the Problem

We need to detect the sequence **"1101"** in a serial bit stream. Key questions:

- What happens when we receive each bit?
- How do we handle **overlapping patterns**? (e.g., "110**1101**" has two "1101" patterns sharing bits)

5.2 Step-by-Step FSM Construction

Pattern to detect: $1 \rightarrow 1 \rightarrow 0 \rightarrow 1$

State	What we've matched so far	Waiting for
S0	Nothing (or mismatch occurred)	First '1'
S1	"1"	Second '1'
S2	"11"	'0'
S3	"110"	Final '1'
S4	"1101" - COMPLETE!	(Output Z=1)

5.3 Tricky Transitions Explained

1. Why does S2 loop on input '1'?

- In S2, we have "11". If we get another '1', we now have "111"
- The last two bits "11" are still a valid partial match for "1101"
- So we stay in S2 (we still have "11" matched)

2. Why does S4 go to S2 on input '1'?

- In S4, we just detected "1101". If next input is '1':
- The sequence "1101**1**" ends with "1" which could start a new "1101"
- Actually, "01" at the end + new "1" = we have "1" matched → but wait!
- "1101" ends with "01", and if we get "1", we have "011" - only last "1" useful
- But "1101" also contains "1" at position 4, so go to S1? No!
- Actually: "1101" + "1" = "11011". Looking backward: "11" is matched → S2

3. Why does S3 go to S0 on input '0'?

- In S3, we have "110". If we get '0', we have "1100"
- No suffix of "1100" matches any prefix of "1101"
- Start over from S0

5.4 Moore Machine: FSM State Diagram

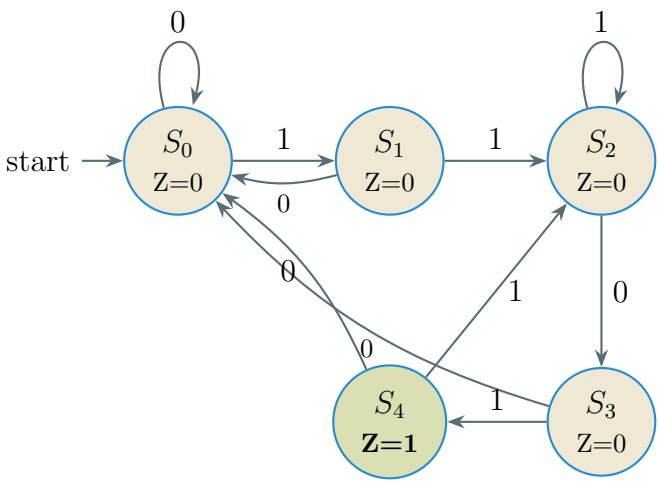

Figure 6: "1101" Sequence Detector - Moore Machine (Output in state)

5.5 Moore: Complete State Transition Table

Current State	Matched Pattern	Input X	Next State	Output Z (Moore)
S0	—	0	S0	0
S0	—	1	S1	0
S1	"1"	0	S0	0
S1	"1"	1	S2	0
S2	"11"	0	S3	0
S2	"11"	1	S2	0
S3	"110"	0	S0	0
S3	"110"	1	S4	0
S4	"1101"	0	S0	**1**
S4	"1101"	1	S2	**1**

5.6 Moore VHDL Implementation

seq_detector_1101_moore.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity seq_detector_1101_moore is
    port (
        clk : in std_logic;
        rst : in std_logic;
        x   : in std_logic;
        z   : out std_logic
    );
end seq_detector_1101_moore;

```

```
architecture bhv of seq_detector_1101_moore is
    -- S0: nothing matched
    -- S1: matched "1"
    -- S2: matched "11"
    -- S3: matched "110"
    -- S4: matched "1101" -> OUTPUT!
    type state_type is (s0, s1, s2, s3, s4);
    signal current_state, next_state : state_type;
begin

------------- Process 1: State Register -------------
    process (clk, rst)
    begin
        if rst = '1' then
            current_state <= s0;
        elsif rising_edge(clk) then
            current_state <= next_state;
        end if;
    end process;

------------- Process 2: Next State Logic -------------
    process (current_state, x)
    begin
        case current_state is
            when s0 =>  -- Nothing matched yet
                if x = '1' then
                    next_state <= s1;  -- Got first '1'
                else
                    next_state <= s0;  -- Stay, wait for '1'
                end if;

            when s1 =>  -- Matched "1"
                if x = '1' then
                    next_state <= s2;  -- Got "11"
                else
                    next_state <= s0;  -- "10" - no match, restart
                end if;

            when s2 =>  -- Matched "11"
                if x = '1' then
                    next_state <= s2;  -- "111" -> still have "11"
                else
                    next_state <= s3;  -- Got "110"
                end if;

            when s3 =>  -- Matched "110"
                if x = '1' then
                    next_state <= s4;  -- Got "1101" - DETECTED!
                else
```

```
                    next_state <= s0;  -- "1100" - no useful suffix
                end if;

            when s4 =>  -- Matched "1101" - OUTPUT STATE
                if x = '1' then
                    next_state <= s2;  -- "11011" ends with "11"
                else
                    next_state <= s0;  -- "11010" - restart
                end if;
        end case;
    end process;

------------- Process 3: Output Logic (Moore) -------------
-- Moore: Output depends ONLY on current state
    process (current_state)
    begin
        case current_state is
            when s0 => z <= '0';
            when s1 => z <= '0';
            when s2 => z <= '0';
            when s3 => z <= '0';
            when s4 => z <= '1';  -- OUTPUT HERE!
        end case;
    end process;

end bhv;
```

5.7 Mealy Machine: FSM State Diagram

In Mealy, output appears on the **transition** (immediately when final '1' arrives), not after entering a state. This means we need **only 4 states** (no S4 needed).

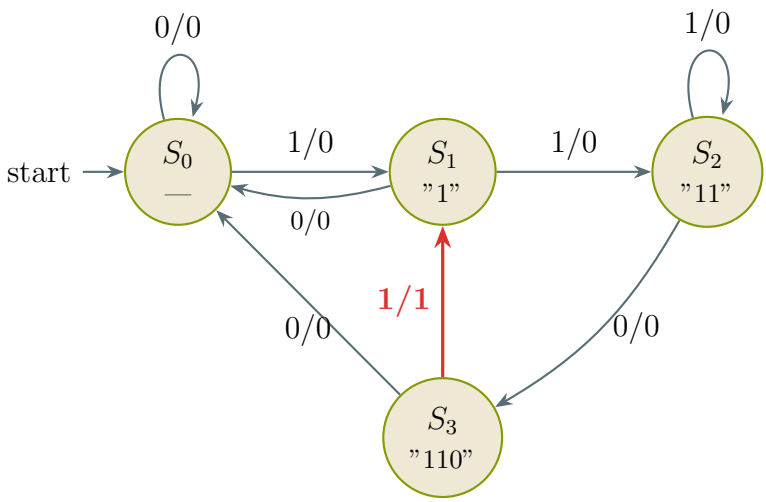

Figure 7: "1101" Sequence Detector - Mealy Machine (Output on transition)

Key Difference: The red arrow shows Z=1 output **immediately** when the final '1' arrives, while transitioning to S1 (since "1101" ends with "1").

5.8 Mealy: Complete State Transition Table

Current State	Matched Pattern	Input X	Next State	Output Z (Mealy)
S0	—	0	S0	0
S0	—	1	S1	0
S1	"1"	0	S0	0
S1	"1"	1	S2	0
S2	"11"	0	S3	0
S2	"11"	1	S2	0
S3	"110"	0	S0	0
S3	"110"	1	S1	**1** ← Immediate!

5.9 Mealy VHDL Implementation

seq_detector_1101_mealy.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity seq_detector_1101_mealy is
    port (
        clk : in std_logic;
        rst : in std_logic;
        x   : in std_logic;
```

```
        z    : out std_logic
    );
end seq_detector_1101_mealy;

architecture bhv of seq_detector_1101_mealy is
    -- Only 4 states needed (no output state)
    -- S0: nothing matched
    -- S1: matched "1"
    -- S2: matched "11"
    -- S3: matched "110"
    type state_type is (s0, s1, s2, s3);
    signal current_state, next_state : state_type;
begin

------------- Process 1: State Register -------------
    process (clk, rst)
    begin
        if rst = '1' then
            current_state <= s0;
        elsif rising_edge(clk) then
            current_state <= next_state;
        end if;
    end process;

------------- Process 2: Next State Logic -------------
    process (current_state, x)
    begin
        case current_state is
            when s0 =>
                if x = '1' then
                    next_state <= s1;
                else
                    next_state <= s0;
                end if;

            when s1 =>
                if x = '1' then
                    next_state <= s2;
                else
                    next_state <= s0;
                end if;

            when s2 =>
                if x = '1' then
                    next_state <= s2;   -- "111" -> "11"
                else
                    next_state <= s3;   -- "110"
                end if;

            when s3 =>
                if x = '1' then
```

```
                    next_state <= s1;  -- "1101" -> "1" (output here!)
                else
                    next_state <= s0;
                end if;
        end case;
    end process;

------------- Process 3: Output Logic (Mealy) -------------
-- Mealy: Output depends on state AND input
    process (current_state, x)
    begin
        z <= '0';  -- Default output
        case current_state is
            when s0 => z <= '0';
            when s1 => z <= '0';
            when s2 => z <= '0';
            when s3 =>
                if x = '1' then
                    z <= '1';  -- IMMEDIATE OUTPUT!
                else
                    z <= '0';
                end if;
        end case;
    end process;

end bhv;
```

6 Timing Comparison: Moore vs Mealy

6.1 Example Input: "1101101"

Moore Machine Timing:

Clock	0	1	2	3	4	5	6	7
Input X	-	1	1	0	1	1	0	1
State	S0	S1	S2	S3	solarizedGreen!30S4	S2	S3	solarizedGreen!30S4
Output Z	0	0	0	0	**1**	0	0	**1**

Mealy Machine Timing:

Clock	0	1	2	3	4	5	6	7
Input X	-	1	1	0	1	1	0	1
State	S0	S1	S2	S3	S1	S2	S3	S1
Output Z	0	0	0	0	**1***	0	0	**1***

*Output appears **during** clock cycle 4, not after.

6.2 Key Observations

1. **Moore**: Output appears in clock cycle 4 (after entering S4)
2. **Mealy**: Output appears during the transition in cycle 4 (immediately with input)
3. **Moore needs 5 states**, Mealy needs only 4 states
4. **Moore output is more stable** (synchronized with clock)
5. **Mealy responds faster** (no extra clock delay)

7 Conclusion

This lab demonstrated:

1. **Moore Machines**: Used for counters and sequence detectors where output stability is important (output changes only on clock edges)
2. **Mealy Machines**: Used when immediate response is needed (output changes asynchronously with input)
3. **FSM Design Pattern**: Three-process architecture (state register, next-state logic, output logic) provides clean separation of concerns
4. **Overlapping Detection**: Critical for sequence detectors - the FSM must transition to an appropriate state to catch consecutive patterns